- **Name:** Asad Ullah
- **Program:** Electrical Engineering (AI)
- **Semester:**2nd
- **Section:** B
- **Registration No:** BF25NWELE0686

Lab:4

Switch, Break, and Continue Statements in C++

The objective of this lab is to:

1. Learn how to use **switch statements** to handle multiple choices based on user input.
2. Understand the role of **break statements** in preventing fall-through in switch cases.
3. Learn how to use **continue statements** in loops to skip certain iterations.
4. Apply these statements to solve practical problems like **day-name selection, grade evaluation, and unit conversion**.

By the end of this lab, students will understand **decision-making and control-flow structures** in C++.

Introduction

In C++ programming, controlling the flow of execution is essential. While **if-else statements** can be used for decision making, they can become cumbersome when handling multiple discrete options.

The **switch statement** provides a cleaner alternative for multi-choice decisions. The **break statement** ensures that only the matching case executes, while the **default case** handles unexpected inputs.

Additionally, **continue statements** are used in loops to skip the current iteration and continue with the next one. These statements make programs more **efficient, readable, and maintainable**.

Theory

1. Switch Statement

The switch statement allows checking a single variable against multiple possible values.

Syntax:

```
switch(expression)
{
    case value1:
        statements;
        break;
    case value2:
        statements;
        break;
    ...
    default:
        statements;
}
```

Key Points:

- The **expression** is evaluated once.
- Each **case** checks for a match with the expression.
- **break** stops the execution after a case is executed.
- **default** executes if none of the cases match.

---

2. Break Statement

- Terminates the current **switch case** or loop immediately.
- Prevents the **fall-through** effect in switch cases.
- Essential for controlling flow in loops and switch structures.

---

3. Continue Statement

- Skips the **current iteration** of a loop and proceeds to the **next iteration**.
- Useful when you want to **ignore certain values** in loops without exiting the loop entirely.

Lab Tasks and Programs

Task 1: Day's Name Using Switch Statement

Problem Statement

- The program prompts the user to enter a number (1–7) representing a day of the week.
- Outputs the corresponding day name using a **switch statement**.
- Uses **break** to prevent fall-through.

Program

```
#include <iostream>
using namespace std;

int main() {
    int day;
    cout << "Enter a day number (1-7): ";
    cin >> day;

    switch(day) {
        case 1:
            cout << "Monday";
            break;
        case 2:
            cout << "Tuesday";
            break;
        case 3:
            cout << "Wednesday";
            break;
        case 4:
            cout << "Thursday";
            break;
        case 5:
            cout << "Friday";
            break;
        case 6:
            cout << "Saturday";
            break;
        case 7:
            cout << "Sunday";
            break;
        default:
            cout << "Invalid input";
    }

    return 0;
}
```

Explanation

- The user inputs a number 1–7.
- The switch statement checks which case matches.

- Each case prints the corresponding day and ends with **break** to prevent executing subsequent cases.
- The **default case** handles invalid input outside 1–7.

Task 2: Grade Evaluation Using Switch Statement

Problem Statement

- Input a **character grade** (A, B, C, D, F).
- Output the corresponding evaluation:

Grade	Result
A	Excellent
B	Very Good
C	Good
D	Pass
F	Fail

- Display **"Invalid Grade"** for any other character.

Program

```
#include <iostream>
using namespace std;

int main() {
    char grade;
    cout << "Enter your grade (A-F): ";
    cin >> grade;

    switch(grade) {
        case 'A':
            cout << "Excellent";
            break;
        case 'B':
            cout << "Very Good";
            break;
        case 'C':
            cout << "Good";
            break;
        case 'D':
            cout << "Pass";
            break;
```

```
        case 'F':
            cout << "Fail";
            break;
        default:
            cout << "Invalid Grade";
    }

    return 0;
}
```

Explanation

- Switch evaluates the **grade character**.
- Each case outputs the evaluation for the corresponding grade.
- **break** prevents execution of other cases.
- **default** handles invalid grades.

Task 3: Unit Conversion Menu Using Switch Statement

Problem Statement

- Display a menu of conversions:
  1 → Kilometers to Meters
  2 → Meters to Centimeters
  3 → Kilograms to Grams
  4 → Celsius to Fahrenheit
- Take user choice and input value.
- Perform the conversion and display result.
- Show "**Invalid choice**" if input is wrong.

Program

```
#include <iostream>
using namespace std;

int main() {
    int choice;
    float value;

    cout << "Conversion Menu:\n";
    cout << "1 -> Kilometers to Meters\n";
    cout << "2 -> Meters to Centimeters\n";
    cout << "3 -> Kilograms to Grams\n";
    cout << "4 -> Celsius to Fahrenheit\n";

    cout << "Enter your choice (1-4): ";
    cin >> choice;

    switch(choice) {
        case 1:
```

```
            cout << "Enter kilometers: ";
            cin >> value;
            cout << value << " km = " << value * 1000 << " meters";
            break;
        case 2:
            cout << "Enter meters: ";
            cin >> value;
            cout << value << " m = " << value * 100 << " cm";
            break;
        case 3:
            cout << "Enter kilograms: ";
            cin >> value;
            cout << value << " kg = " << value * 1000 << " grams";
            break;
        case 4:
            cout << "Enter Celsius: ";
            cin >> value;
            cout << value << "°C = " << (value * 9/5) + 32 << "°F";
            break;
        default:
            cout << "Invalid choice";
    }

    return 0;
}
```

Explanation

- Menu is displayed using **cout**.
- User enters **choice (1–4)**.
- Switch evaluates choice and performs the corresponding conversion.
- **break** prevents executing other cases.
- **default** handles invalid choices.

Conversions implemented:

- 1 km = 1000 m
- 1 m = 100 cm
- 1 kg = 1000 g
- Celsius → Fahrenheit: F=C×9/5+32F = C × 9/5 + 32F=C×9/5+32

In Lab-04, we learned:

1. How to use **switch statements** to handle multiple choices efficiently.
2. The importance of **break statements** in preventing fall-through.

3. How **default case** handles unexpected input.
4. The **continue statement** can be used in loops to skip certain iterations (though not used in these tasks).
5. Applying switch statements to **real-world problems** like day selection, grade evaluation, and unit conversion.

This lab enhanced understanding of **decision-making and control-flow in C++**, which is a core concept in programming.